

CropCare AI • Production Engineering Guide

A step-by-step implementation manual to deploy a production-grade, rural-resilient AI crop health assistant for smallholder farmers.

Version 2.0 • September 2026 • Smart India Hackathon / AgTech Specification

Table of Contents (7 Core Modules)

1. **Module 1:** Modernize Frontend Build & Offline PWA (Essential for Rural Connectivity)
2. **Module 2:** Connect Real AI/ML Models (Edge + Cloud Inference)
3. **Module 3:** Replace Mock Weather with a Live Meteorological API
4. **Module 4:** Production Database, Storage & Mobile OTP Auth
5. **Module 5:** Verified Agronomic Database (CIBRC / KVK Compliance)
6. **Module 6:** Voice Input/Output (Bhashini AI / Web Speech API)
7. **Module 7:** Deployment & DevOps Checklist

1. Modernize Frontend Build & Offline PWA

Target Goal: Transform the frontend into an installable Progressive Web App (PWA) that loads in under 200 milliseconds, functions 100% offline in rural farm plots, and requires zero app store approvals.

WHY THIS MATTERS FOR SMALL FARMERS

In rural areas, farmers frequently experience 2G/3G connections or complete signal blackouts in the middle of their field. A traditional website that downloads a heavy bundle every visit will fail. By using Vite + Service Workers, the entire application shell, diagnosis library, and offline crop doctor remain fully functional without internet.

Step 1: Scaffolding with Vite + React + TypeScript

Open your terminal and run the standard Vite initializer. This sets up lightning-fast Hot Module Replacement (HMR) and production bundling:

```
# 1. Create the project
npm create vite@latest cropcare-pwa -- --template react-ts

# 2. Enter directory and install core dependencies
cd cropcare-pwa
npm install lucide-react recharts idb-keyval

# 3. Install Tailwind CSS and PWA plugin
npm install -D tailwindcss postcss autoprefixer vite-plugin-pwa
npx tailwindcss init -p
```

Step 2: Configure vite.config.ts with PWA Plugin

Replace your vite.config.ts with the following configuration. It sets up automatic Service Worker caching and an installable Web Manifest:

```

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['favicon.ico', 'apple-touch-icon.png', 'leaf-icon.svg'],
      manifest: {
        name: 'CropCare AI - Farmer Health Assistant',
        short_name: 'CropCare AI',
        description: 'AI Crop Disease Detection & Care for Small Farmers',
        theme_color: '#123524',
        background_color: '#F5F7F3',
        display: 'standalone',
        orientation: 'portrait',
        start_url: '/',
        icons: [
          {
            src: 'pwa-192x192.png',
            sizes: '192x192',
            type: 'image/png'
          },
          {
            src: 'pwa-512x512.png',
            sizes: '512x512',
            type: 'image/png',
            purpose: 'any maskable'
          }
        ]
      }
    ]
  },
  workbox: {
    // Cache all local static assets
    globPatterns: ['**/*.{js,css,html,ico,png,svg,woff2}'],
    runtimeCaching: [
      {
        // Cache Google Fonts for offline use
        urlPattern: /^https:\/\/fonts\.googleapis\.com\/.*\/i/,
        handler: 'CacheFirst',
        options: {
          cacheName: 'google-fonts-cache',
          expiration: { maxEntries: 10, maxAgeSeconds: 60 * 60 * 24 * 365 }
        }
      },
      {
        // Cache weather API with network fallback
        urlPattern: /^https:\/\/api\.open-meteo\.com\/.*\/i/,
        handler: 'NetworkFirst',
        options: {
          cacheName: 'weather-cache',
          networkTimeoutSeconds: 3,
          expiration: { maxEntries: 20, maxAgeSeconds: 60 * 60 * 4 } // 4 hours
        }
      }
    ]
  }
});

```

Step 3: Storing Scans Offline via IndexedDB (idb-keyval)

Do not use `localStorage` for leaf photos because it has a 5MB limit. Use IndexedDB so the farmer can save hundreds of offline high-resolution leaf scans directly on device:

```
// src/services/offlineStorage.ts
import { get, set, entries } from 'idb-keyval';

export interface OfflineScanRecord {
  id: string;
  timestamp: number;
  crop: string;
  imageBlob: Blob;
  predictedDisease?: string;
  severity?: number;
  synced: boolean;
}

export async function saveOfflineScan(scan: OfflineScanRecord) {
  await set(`scan_${scan.id}`, scan);
  console.log('Saved scan offline in IndexedDB:', scan.id);
}

export async function getAllOfflineScans(): Promise<OfflineScanRecord[]> {
  const all = await entries();
  return all
    .filter(([k]) => typeof k === 'string' && k.startsWith('scan_'))
    .map([, v]) => v as OfflineScanRecord;
}
```

Step 4: Build & Test Verification

1. Run `npm run build`. Notice the output creates a `dist/sw.js` Service Worker file.
2. Run `npm run preview` to test the production build locally.
3. Open Chrome DevTools $\rightarrow$ **Application** Tab $\rightarrow$ **Service Workers**. Verify status is *Active and running*.
4. Check the **Offline** checkbox in DevTools Network tab and reload the page. The app must load immediately!

2. Connect Real AI/ML Models (Edge + Cloud)

Target Goal: Replace simulated AI data with real deep learning models: an offline edge classifier using ONNX Web for zero-connectivity fields, and a cloud-based FastAPI YOLOv8 service for bounding-box disease localization.

Step 1: Dataset & Model Strategy

Model Type	Architecture	Dataset Sources	Latency & Size
Disease Classifier (Edge)	MobileNetV3-Large / EfficientNet-B0	PlantVillage (54k images) + ICAR field samples	<150ms on mobile CPU (~12 MB INT8)
Lesion & Pest Detector (Cloud)	YOLOv8-nano (<code>`yolov8n.pt`</code>)	Roboflow Agricultural Disease Dataset (annotated boxes)	~80ms on Cloud GPU / ~300ms CPU

Step 2: Production FastAPI Service (`backend/main.py`)

Create your Python backend using FastAPI and Ultralytics YOLO:

```

# backend/main.py
from fastapi import FastAPI, UploadFile, File, Form, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from PIL import Image
import io
import imagehash
from ultralytics import YOLO

app = FastAPI(title="CropCare AI Inference Engine", version="2.0")

# Enable CORS for frontend domain
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Load pre-trained / fine-tuned YOLO model
model = YOLO("yolov8n-crop-diseases.pt")

# In-memory duplicate hash store (replace with Redis/Postgres in Step 4)
SEEN_HASHES = set()

@app.post("/api/v1/scan")
async def scan_crop(file: UploadFile = File(...), crop: str = Form("Tomato")):
    if not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="Uploaded file is not an image")

    contents = await file.read()
    image = Image.open(io.BytesIO(contents)).convert("RGB")

    # 1. Duplicate Image Detection via Perceptual Hash
    img_hash = str(imagehash.dhash(image))
    is_duplicate = img_hash in SEEN_HASHES
    SEEN_HASHES.add(img_hash)

    # 2. Run YOLO Object Detection for Disease Lesions
    results = model(image)
    detections = []
    total_area_pixels = image.width * image.height
    affected_pixels = 0

    for r in results:
        for box in r.boxes:
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            conf = float(box.conf[0])
            cls_id = int(box.cls[0])
            name = model.names[cls_id]

            box_area = (x2 - x1) * (y2 - y1)
            affected_pixels += box_area

            detections.append({
                "disease": name,
                "confidence": round(conf * 100, 1),
                "bbox": [round(x1), round(y1), round(x2), round(y2)]
            })

    affected_percentage = min(100.0, round((affected_pixels / total_area_pixels) * 100, 1))

    return {

```

```

        "status": "success",
        "is_duplicate": is_duplicate,
        "image_hash": img_hash,
        "crop": crop,
        "detections_count": len(detections),
        "detections": detections,
        "severity_score": min(100, int(affected_percentage * 1.4)),
        "affected_area_percentage": affected_percentage
    }

```

Step 3: Exporting Model to ONNX for 100% Offline Browser Inference

To run the model directly inside a farmer's Chrome browser with zero internet, export and quantize your model to ONNX format:

```

# Run in Python to create lightweight web model
import torch
import torchvision.models as models

# Example: Export MobileNetV3
model = models.mobilenet_v3_small(pretrained=True)
dummy_input = torch.randn(1, 3, 224, 224)

torch.onnx.export(
    model, dummy_input, "cropcare_mobilenet.onnx",
    input_names=['input'], output_names=['output'],
    opset_version=13
)
print("Exported cropcare_mobilenet.onnx")

```

Step 4: Frontend In-Browser Inference via `onnxruntime-web`

Install `onnxruntime-web` in your frontend:

```
npm install onnxruntime-web
```

```

// src/services/edgeInference.ts
import * as ort from 'onnxruntime-web';

export async function runLocalLeafDiagnosis(imageElement: HTMLImageElement) {
    // Load model cached by PWA Service Worker
    const session = await ort.InferenceSession.create('/models/cropcare_mobilenet.onnx', {
        executionProviders: ['wasm']
    });

    // Preprocess image tensor to 224x224 normalized RGB
    const tensor = preprocessImageToTensor(imageElement);
    const feeds = { input: tensor };
    const results = await session.run(feeds);
    const output = results.output.data;

    return parseProbabilities(output);
}

```

3. Connect Live Meteorological Weather API

Target Goal: Feed real-time microclimate metrics (relative humidity, rainfall, dew-point, wind) directly into the disease spread algorithm using the free Open-Meteo API.

WHY WEATHER DRIVES DISEASE SPREAD

Fungal spores (such as *Alternaria solani* and *Phytophthora infestans*) cannot germinate on dry leaves. They require $\geq 80\%$ relative humidity or standing water droplets for 12 to 24 consecutive hours. Connecting hyper-local weather turns a static diagnosis into an early warning radar.

Step 1: The Open-Meteo API Endpoint

The Open-Meteo API is 100% free for commercial & non-commercial use, requires no API key, and offers 1km spatial resolution:

```
GET https://api.open-meteo.com/v1/forecast?latitude=22.7196&longitude=75.8577&hourly=temperature_2m,rel
```

Step 2: Frontend Weather Service (`src/services/weatherService.ts`)

```
// src/services/weatherService.ts
export interface LiveWeatherReport {
  temperature: number;
  humidity: number;
  rainProbability: number;
  windSpeed: number;
  fungalRiskLevel: 'Low' | 'Moderate' | 'Critical';
  advisory: string;
}

export async function fetchFieldWeather(lat: number, lng: number): Promise<LiveWeatherReport> {
  const url = `https://api.open-meteo.com/v1/forecast?latitude=${lat}&longitude=${lng}&hourly=temperature,relativehumidity,precipitationprobability,windspeed`;

  const res = await fetch(url);
  if (!res.ok) throw new Error('Weather API fetch failed');
  const data = await res.json();

  const currentTemp = data.current_weather.temperature;
  const currentWind = data.current_weather.windspeed;
  const currentHumidity = data.hourly.relativehumidity_2m[0];
  const currentRainProb = data.hourly.precipitation_probability[0] || 0;

  // Scientific Fungal Spore Germination Risk Logic
  let risk: 'Low' | 'Moderate' | 'Critical' = 'Low';
  let advisory = 'Environmental conditions are stable for crop growth.';

  if (currentHumidity ≥ 85 && currentRainProb ≥ 60) {
    risk = 'Critical';
    advisory = 'High humidity (>85%) and rain expected. Critical risk of fungal spore germination!';
  } else if (currentHumidity ≥ 75 || currentRainProb ≥ 40) {
    risk = 'Moderate';
    advisory = 'Elevated leaf moisture detected. Inspect lower canopy foliage.';
  }

  return {
    temperature: currentTemp,
    humidity: currentHumidity,
    rainProbability: currentRainProb,
    windSpeed: currentWind,
    fungalRiskLevel: risk,
    advisory
  };
}
```

Step 3: Calculating 7-Day Disease Spread Progression

Use this mathematical model to project how severity expands over 7 days based on current humidity:

```
// Calculates 7-day severity trajectory
export function calculateSpreadTrajectory(baseSeverity: number, humidity: number, rainProb: number) {
  // Humidity multiplier: >80% humidity accelerates growth by up to 1.8x
  const humidityFactor = humidity > 80 ? 1 + (humidity - 80) * 0.03 : 1.0;
  const rainFactor = 1 + (rainProb / 100) * 0.2;

  const trajectory = [];
  let current = baseSeverity;

  for (let day = 1; day ≤ 7; day++) {
    const dailyGrowth = current * 0.08 * humidityFactor * rainFactor;
    current = Math.min(95, Math.round(current + dailyGrowth));
    trajectory.push({ day: `Day ${day}`, severity: current });
  }

  return trajectory;
}
```

4. Production Database, Storage & Mobile OTP

Target Goal: Establish persistent farmer accounts, geospatial field boundaries, compressed cloud photo storage, and seamless phone OTP login.

Step 1: Production Database Schema (PostgreSQL + PostGIS)

Run these DDL statements in Supabase, Neon, or your PostgreSQL instance:

```
-- Enable PostGIS for field boundary coordinates
CREATE EXTENSION IF NOT EXISTS postgis;

-- 1. Farmers Table
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  mobile_number VARCHAR(15) UNIQUE NOT NULL,
  full_name VARCHAR(100) NOT NULL,
  village VARCHAR(100),
  district VARCHAR(100),
  state VARCHAR(100),
  preferred_language VARCHAR(10) DEFAULT 'hi',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 2. Farms & Fields
CREATE TABLE fields (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  field_name VARCHAR(100) NOT NULL,
  crop VARCHAR(50) NOT NULL,
  variety VARCHAR(100),
  area_acres NUMERIC(5,2) NOT NULL,
  sowing_date DATE,
  boundary_geom GEOMETRY(POLYGON, 4326), -- GPS Field Polygon
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 3. Scans & AI Diagnoses
CREATE TABLE crop_scans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  field_id UUID REFERENCES fields(id) ON DELETE CASCADE,
  image_url TEXT NOT NULL,
  image_phash VARCHAR(64) NOT NULL,
  severity_percentage NUMERIC(5,2) NOT NULL,
  affected_area_percentage NUMERIC(5,2) NOT NULL,
  detected_diseases JSONB NOT NULL,
  weather_snapshot JSONB,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 4. Index for fast duplicate image lookups
CREATE INDEX idx_scans_phash ON crop_scans(image_phash);
```

Step 2: Passwordless Mobile OTP Login with Firebase Auth

Small farmers do not use passwords. Use Firebase Phone Auth for instant SMS OTP verification:

```
// src/services/phoneAuth.ts
import { getAuth, RecaptchaVerifier, signInWithPhoneNumber } from 'firebase/auth';

// 1. Initialize invisible reCAPTCHA
export function setupRecaptcha(buttonId: string) {
  const auth = getAuth();
  return new RecaptchaVerifier(auth, buttonId, {
    size: 'invisible'
  });
}

// 2. Send SMS OTP
export async function sendOtpToMobile(phoneNumber: string, appVerifier: any) {
  const auth = getAuth();
  const confirmationResult = await signInWithPhoneNumber(auth, `+91${phoneNumber}`, appVerifier);
  return confirmationResult; // Call confirmationResult.confirm(otpCode) on user input
}
```

Step 3: Client-Side WebP Compression & Cloudflare R2 Upload

Never upload raw 5MB camera photos over rural 3G networks. Compress images to lightweight WebP (<200KB) in the browser before sending:

```
// Compresses canvas image to WebP blob
export async function compressLeafImage(file: File): Promise<Blob> {
  return new Promise((resolve) => {
    const img = new Image();
    img.src = URL.createObjectURL(file);
    img.onload = () => {
      const canvas = document.createElement('canvas');
      const MAX_WIDTH = 1200;
      const scale = MAX_WIDTH / img.width;
      canvas.width = MAX_WIDTH;
      canvas.height = img.height * scale;

      const ctx = canvas.getContext('2d')!;
      ctx.drawImage(img, 0, 0, canvas.width, canvas.height);

      canvas.toBlob((blob) => resolve(blob!), 'image/webp', 0.82);
    };
  });
}
```

5. Verified Agronomic Database (CIBRC/KVK)

Target Goal: Ensure all pesticide, fungicide, and bio-agent remedies strictly adhere to government-registered guidelines (CIBRC / ICAR / State Agricultural Universities).

Warning: Never allow an LLM or unverified AI model to invent pesticide dosages. Chemical misuse can destroy crops, pollute groundwater, and violate the Insecticides Act, 1968. Every treatment recommendation must pull from a verified database.

Step 1: CIBRC-Compliant Database Table Structure

Field	Type	Description & Example
target_crop	VARCHAR(50)	Crop name (e.g. Tomato)
target_disease	VARCHAR(100)	Pathogen or pest (e.g. Early Blight)
chemical_type	VARCHAR(30)	Organic, Biological, or Chemical
active_ingredient	VARCHAR(100)	Copper Oxychloride 50% WP
dose_per_acre	VARCHAR(50)	500 - 600 grams
dilution_water_liters	INTEGER	200 Liters / Acre
pre_harvest_interval_days	INTEGER	Waiting period before harvest (e.g. 3 days)
banned_status	BOOLEAN	Must be FALSE (checked against CIBRC banned list)

Step 2: Seed Dataset JSON (data/cibrc_treatments.json)

```
[
  {
    "crop": "Tomato",
    "disease": "Early Blight",
    "severity_threshold": "Moderate",
    "cultural_control": "Prune lower infected leaves; sanitize pruning tools with 1% sodium hypochlorite",
    "biological_control": "Trichoderma harzianum @ 5g/L foliar spray during early symptoms.",
    "chemical_control": {
      "product_name": "Copper Oxychloride 50% WP",
      "dosage": "2.5 g / Liter of water (500g in 200L water per acre)",
      "method": "Thorough foliar spray on both leaf surfaces at morning hours",
      "repeat_interval_days": 10,
      "pre_harvest_interval_days": 3,
      "cibrc_registration_valid": true
    },
    "safety_precautions": "Wear mask and gloves. Avoid spraying when wind speed > 15 km/h.",
    "helpline": "Kisan Call Centre: 1800-180-1551"
  },
  {
    "crop": "Tomato",
    "disease": "Fruit Borer",
    "severity_threshold": "Larval Emergence",
    "cultural_control": "Erect 4-6 Helilure pheromone traps per acre at crop canopy level.",
    "biological_control": "Azadirachtin 10,000 ppm (Neem oil biopesticide) @ 2 ml / Liter of water.",
    "chemical_control": {
      "product_name": "Chlorantraniliprole 18.5% SC",
      "dosage": "0.3 ml / Liter of water (60 ml per acre in 200L water)",
      "method": "Target flower bunches and young fruiting shoots",
      "repeat_interval_days": 14,
      "pre_harvest_interval_days": 3,
      "cibrc_registration_valid": true
    },
    "safety_precautions": "Safe for pollinators when applied post-sunset. Toxic to fish; keep away from",
    "helpline": "Kisan Call Centre: 1800-180-1551"
  }
]
```

6. Voice Input/Output (Bhashini / Web Speech)

Target Goal: Enable farmers to speak their crop questions and hear answers spoken aloud in their native tongue without typing.

WHY VOICE IS ESSENTIAL FOR SMALL FARMERS

Many smallholder farmers have limited formal literacy or have wet, muddy hands while inspecting crops in the field. Voice interaction makes CropCare AI as natural as talking to an agricultural expert standing beside them.

Method A: Zero-Cost Native Web Speech API (Works on all Android Phones)

No backend, server, or API key needed. Runs directly on device:

```
// src/hooks/useVoiceAssistant.ts
import { useState, useEffect } from 'react';

export function useVoiceAssistant(languageCode = 'hi-IN') {
  const [isListening, setIsListening] = useState(false);
  const [transcript, setTranscript] = useState('');

  // 1. Speech-to-Text (Farmer Speaking)
  const startListening = () => {
    const SpeechRecognition = (window as any).SpeechRecognition || (window as any).webkitSpeechRecognition;
    if (!SpeechRecognition) {
      alert('Speech recognition is not supported in this browser.');
```

```
      return;
    }

    const recognition = new SpeechRecognition();
    recognition.lang = languageCode; // e.g. 'hi-IN', 'mr-IN', 'te-IN', 'ta-IN'
    recognition.continuous = false;
    recognition.interimResults = false;

    recognition.onstart = () => setIsListening(true);
    recognition.onend = () => setIsListening(false);
    recognition.onresult = (event: any) => {
      const text = event.results[0][0].transcript;
      setTranscript(text);
    };

    recognition.start();
  };

  // 2. Text-to-Speech (AI Doctor Speaking Back)
  const speakText = (text: string) => {
    if (!('speechSynthesis' in window)) return;
    window.speechSynthesis.cancel(); // Stop ongoing speech

    const utterance = new SpeechSynthesisUtterance(text);
    utterance.lang = languageCode;
    utterance.rate = 0.95; // Slightly slower for clear agricultural comprehension
    window.speechSynthesis.speak(utterance);
  };

  return { isListening, transcript, startListening, speakText };
}
```

Method B: Digital India's Bhashini AI (Government Open Speech Engine)

For rural dialects, Government of India's **Bhashini AI** offers the highest accuracy across Indian languages:

```
# backend/bhashini_service.py
import requests

BHASHINI_ENDPOINT = "https://dhruva-api.bhashini.gov.in/services/inference/pipeline"
API_KEY = "YOUR_BHASHINI_API_KEY"

def translate_and_speak(text: str, source_lang: str = "en", target_lang: str = "hi"):
    headers = {
        "Authorization": API_KEY,
        "Content-Type": "application/json"
    }
    payload = {
        "pipelineTasks": [
            {
                "taskType": "translation",
                "config": {
                    "language": { "sourceLanguage": source_lang, "targetLanguage": target_lang }
                }
            },
            {
                "taskType": "tts",
                "config": {
                    "language": { "sourceLanguage": target_lang },
                    "gender": "female"
                }
            }
        ],
        "inputData": { "input": [{ "source": text }] }
    }
    response = requests.post(BHASHINI_ENDPOINT, json=payload, headers=headers)
    return response.json()
```

7. Deployment & DevOps Checklist

Target Goal: Deploy the frontend PWA, FastAPI AI backend, and managed database to production with SSL, CDN caching, and automated scaling.

Step 1: Frontend Deployment to Vercel or Cloudflare Pages (Free Tier)

1. Push your code to GitHub: `git init && git add . && git commit -m "Initial commit" && git push`
2. Log into [Vercel.com](https://vercel.com) or Cloudflare Pages.
3. Click **New Project** and import your GitHub repository.
4. Build settings:
 - **Framework Preset:** Vite
 - **Build Command:** `npm run build`
 - **Output Directory:** `dist`
5. Click **Deploy**. Your PWA is live with automated global SSL within 60 seconds!

Step 2: Backend Containerization with Docker

Save this `Dockerfile` in your backend directory:

```
# backend/Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies for OpenCV and image processing
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgl1-mesa-glx libglib2.0-0 curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Run as non-root user for security
USER 1000

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 3: Deploy Backend on Render.com or Railway

1. Log into [Render.com](#) $\rightarrow$ **New Web Service**.

2. Connect your backend repository and select **Docker runtime**.

3. Configure environment variables:
 - `DATABASE_URL=postgresql://user:password@host/cropcare`
 - `ALLOWED_ORIGINS=https://your-frontend-domain.com`
 - `MODEL_PATH=/app/yolov8n-crop-diseases.pt`

4. Deploy. You receive an automated HTTPS endpoint: `https://cropcare-api.onrender.com`.

Step 4: Production Pre-Flight Checklist

Task	Status	Validation Check
HTTPS & SSL	Critical	PWA Service Workers & Camera access require active SSL (https://).
CORS Security	Critical	Ensure <code>allow_origins</code> in FastAPI matches your production frontend URL only.
Rate Limiting	High	Install <code>slowapi</code> on FastAPI to limit scans to 30 requests / minute per IP.
Lighthouse PWA Audit	High	Run Chrome Lighthouse. Target ≥ 90 on Performance and PWA criteria.
Error Tracking (Sentry)	Medium	Install <code>@sentry/react</code> to receive automated alerts if farmer scans crash.

